

Aktiviteti 5 — Switch Statement

Udhëzues

Deri tani kemi përdorur `if`, `else if` dhe `else` për të kontrolluar kushte të ndryshme. Por kur duhet të kontrollojmë të njëjtën variabël kundrejt shumë vlerave të mundshme (p.sh. dita e javës, një opsion nga menyuja, një operator matematikor), kodi me shumë `else if` bëhet i gjatë dhe i vështirë për t'u lexuar. `switch` statement na jep një mënyrë më të pastër për ta bërë këtë.

Ky udhëzues shpjegon konceptet dhe hapat që do t'i ndjekim, në rendin e duhur.

1. Pse na duhet Switch Statement?

`switch` bën pjesë tek deklaratat kushtëzuese (si `if/else`), por është krijuar posaçërisht për rastin kur duam të krahasojmë **një variabël të vetme** me **shumë vlera të mundshme**.

```
// Me if/else – bëhet e gjatë kur rastet shtohen
let dita = "e Marte";

if (dita === "e Hene") {
  console.log("Fillim java");
} else if (dita === "e Marte") {
  console.log("Dita e dyte");
} else if (dita === "e Merkure") {
  console.log("Mesi i javes");
} else {
  console.log("Diten tjeter");
}
```

`switch` ekzekuton komanda të ndryshme varësisht se cili rast (`case`) plotësohet — pra është i vërtetë.

2. Switch Statement — Sintaksa

```
switch (variabla) {  
  case vlera1:  
    // kodi qe ekzekutohet nese variabla === vlera1  
    break;  
  case vlera2:  
    // kodi qe ekzekutohet nese variabla === vlera2  
    break;  
  default:  
    // kodi qe ekzekutohet nese asnje case nuk perputhet  
}
```

- **variabla** — vlera që kontrollohet brenda **switch** -it.
- **case vlera1:** — rasti i parë brenda switch.
- **case vlera2:** — rasti i dytë, e kështu me radhë.

Shembull:

```
let dita = "e Marte";  
  
switch (dita) {  
  case "e Hene":  
    console.log("Fillim jave");  
    break;  
  case "e Marte":  
    console.log("Dita e dyte");  
    break;  
  case "e Merkure":  
    console.log("Mesi i javes");  
    break;  
  default:  
    console.log("Diten tjeter");  
}
```

💡 **switch** krahason vlerat me **===** (strict equality) — pra kontrollon edhe vlerën edhe tipin, njësoj si **if** -i kur përdorim **===** .

3. Break

Komanda **break** nxjerr ekzekutimin e kodit jashtë trupit të **switch** . Sapo plotësohet një **case** , ekzekutohet kodi brenda tij dhe **switch** përfundon ekzekutimin.

```
let numri = 2;

switch (numri) {
  case 1:
    console.log("Nje");
    break;
  case 2:
    console.log("Dy");
    break; // <-- pa kete, ekzekutimi do te vazhdonte tek case 3
  case 3:
    console.log("Tre");
    break;
}
// -> "Dy"
```

⚠ **Fall-through:** Nëse harrojmë `break`, JavaScript **nuk ndalet** te case-i që u plotësua — vazhdon të ekzekutojë kodin e case-ve në vijim, derisa të hasë një `break` ose të mbarojë `switch` -i. Ky është gabimi më i shpeshtë me `switch`, prandaj çdo `case` (përveç rastit të grupuar, shih pikën 5) duhet të përfundojë me `break`.

4. Default

`default` ekzekutohet atëherë kur asnjëri nga rastet (`case`) nuk është i vërtetë — është ekuivalenti i `else` -s tek `if/else`.

```
let numri = 9;

switch (numri) {
  case 1:
    console.log("Nje");
    break;
  case 2:
    console.log("Dy");
    break;
  default:
    console.log("Nuk e njoh kete numer");
}
// -> "Nuk e njoh kete numer"
```

💡 Vërehet një gjë tek `default`: nuk ka nevojë për `break`, sepse është rasti i fundit dhe menjëherë pasi ta ekzekutojë kodin brenda tij, `switch` -i përfundon vetvetiu. E vendosim zakonisht (por jo domosdoshmërisht) si rasti i fundit brenda `switch`.

5. Disa Cases së bashku (Grouped Cases)

Kur duam të marrim **të njëjtin veprim** për disa `case`, mjafton t'i vendosim njërin pas tjetrit, pa kod ndërmjet tyre — kështu "bien" (fall-through) tek i njëjti bllok kodi.

```
let dita = "Shtune";

switch (dita) {
  case "Hene":
  case "Marte":
  case "Merkure":
  case "Enjte":
  case "Premte":
    console.log("Dite pune");
    break;
  case "Shtune":
  case "Diele":
    console.log("Fundjave");
    break;
}
// -> "Fundjave"
```

💡 Ky është i vetmi rast kur e lëmë fall-through-in **me qëllim** — grupimi i disa case-ve që duan të njëjtin rezultat.

6. Ushtrime praktike (nga sllajdet)

Ushtrimi 1 — Kalkulatori me Switch

1. Kërkoni dy numra nga useri përmes `prompt`, dhe ruajini ata numra në variabla.
2. Kërkoni që useri të japë njërin nga operatorët: mbledhje, zbritje, shumëzim, pjesëtim.
3. Në bazë të operatorit që është kërkuar, përdorni `switch` për të shfaqur rezultatin në `alert`.

```

let numri1 = Number(prompt("Jepni numrin e pare:"));
let numri2 = Number(prompt("Jepni numrin e dyte:"));
let operatori = prompt("Zgjidhni: mbledhje, zbritje, shumezim, pjesetim");

switch (operatori) {
  case "mbledhje":
    alert(numri1 + numri2);
    break;
  case "zbritje":
    alert(numri1 - numri2);
    break;
  case "shumezim":
    alert(numri1 * numri2);
    break;
  case "pjesetim":
    alert(numri1 / numri2);
    break;
  default:
    alert("Operator i panjohur");
}

```

Ushtrimi 2 — Kontrolli i tipit të variablave

Jepen variablat:

```

let a = "test";
let b = 12;
let c;
let d = true;
let e = +d;

```

Kontrolloni tipin e secilës variabël (me `typeof`) dhe, duke përdorur `switch`, shfaqni në konsollë tekstin `"Kjo variabel eshte " + tipi`.

```
let variablat = [a, b, c, d, e];

variablat.forEach((v) => {
  switch (typeof v) {
    case "string":
      console.log("Kjo variabel eshte string");
      break;
    case "number":
      console.log("Kjo variabel eshte number");
      break;
    case "boolean":
      console.log("Kjo variabel eshte boolean");
      break;
    case "undefined":
      console.log("Kjo variabel eshte undefined");
      break;
    default:
      console.log("Tip i panjohur");
  }
});
```

💡 Vini re: `e = +d` — operatori unar `+` e kthen `true` në numër (`1`), koncept që e kemi parë tek ora e operatorëve.

Switch vs if/else — kur përdorim cilin?

- Përdor `switch` kur kontrollon **një variabël të vetme** kundrejt **shumë vlerave të sakta** (equality, `===`).
- Përdor `if/else` kur kushtet janë **krahasime** (`>` , `<` , `&&` , `||`) ose kur kontrollon variabla/kushte të ndryshme nga njëra-tjetra.

Përmbledhje e shpejtë

- | | |
|--|---|
| 1. <code>switch (variabla) { case v: ... break; }</code> | → krahason variablen me vlera te sakta |
| 2. <code>break</code> | → ndalon ekzekutimin, del nga switch |
| 3. pa <code>break</code> | → fall-through, vazhdon te case-i tjetër |
| 4. <code>default</code> | → ekzekutohet nese asnje case nuk perputhet |
| 5. <code>case v1: case v2: ...</code> | → grupim i disa rasteve per te njejtin veprim |

Fjalë të shkurtra

- **Switch statement** — deklaratë kushtëzuese që krahason një variabël me disa vlera të mundshme (`case`)
- **Case** — një rast/vlerë e mundshme brenda `switch` -it
- **Break** — komandë që ndalon ekzekutimin dhe del nga `switch` -i
- **Default** — rasti që ekzekutohet kur asnjë `case` nuk përputhet (ekuivalenti i `else` -s)
- **Fall-through** — kur ekzekutimi "bie" nga një `case` tek tjetri sepse mungon `break` (i padëshiruar, përveç kur grupojmë case-e me qëllim)

Sfida jote (pikë ekstra) — jepet si detyrë shtëpie

Të jepet numri i pikëve nga lënda e matematikës duke përdorur `prompt`, dhe duke përdorur `switch`, të shfaqet nota përkatëse në console:

- Pikët **mbi 80–100** → Nota **5**
- Pikët **mbi 65–80** → Nota **4**
- Pikët **mbi 60–65** → Nota **3**
- Pikët **mbi 50** → Nota **2**
- Pikët **më pak se 50** → `"Na vjen keq, por keni deshtuar ne test"`

 Vini re: `switch` krahason vetëm me barazi të saktë (`===`), jo me intervale (`>` , `<`). Për të zgjidhur këtë sfidë me `switch`, mendoni si mund ta "ndani" numrin në një interval (p.sh. duke e pjesëtuar me 10 dhe duke përdorur `Math.floor`), në mënyrë që çdo interval të bëhet një `case` i vetëm.